

Lect No.(1-2)

Introduction to Web Development

Web development is the process of **creating, designing, and maintaining websites and web applications** for the internet. It combines **programming, design, and deployment** to deliver **interactive, functional, and visually appealing websites**.

Modern web development involves multiple **layers, technologies, and roles** that work together to ensure a seamless experience for the end-user.

Fundamentals of Web Architecture

Web architecture defines the **structure and organization** of a web application. It describes how **clients, servers, databases, and services** interact to deliver content efficiently.

The Client-Server Model

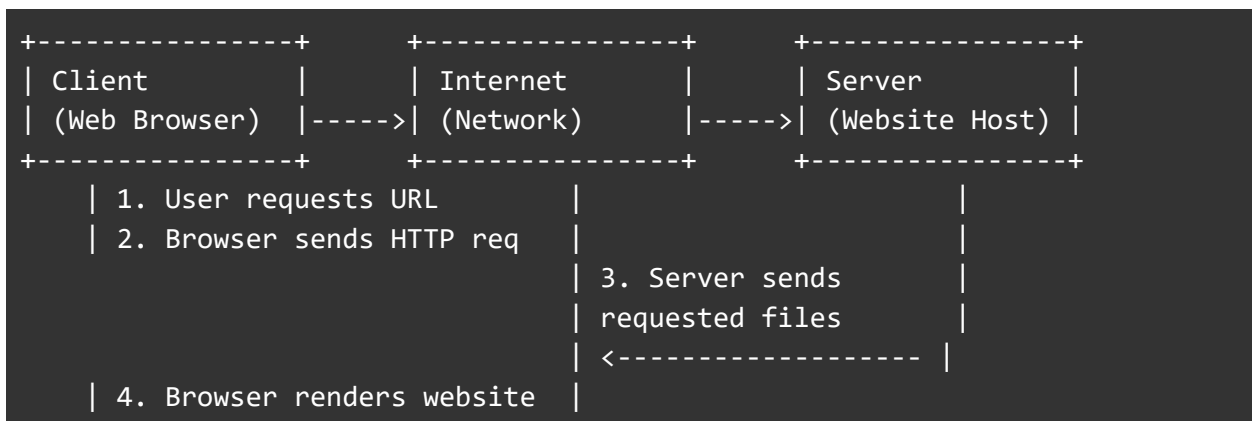
The Client-Server Model is the foundational concept of the web:

- **Client:** Usually a web browser that sends requests for data or content.
- **Server:** Responds to client requests by sending requested data (HTML pages, JSON, files).
- **Internet/Network:** Acts as the communication channel between client and server.

Steps of Interaction:

- 1 User types a URL or clicks a link in the browser.
- 2 Browser sends an HTTP request to the server.
- 3 Server processes the request and sends back the requested files.
- 4 Browser renders the page to the user.

Text Diagram:



Example: Opening www.google.com

Browser requests homepage → Server sends HTML/CSS/JS → Browser renders content

The Three-Tier Architecture

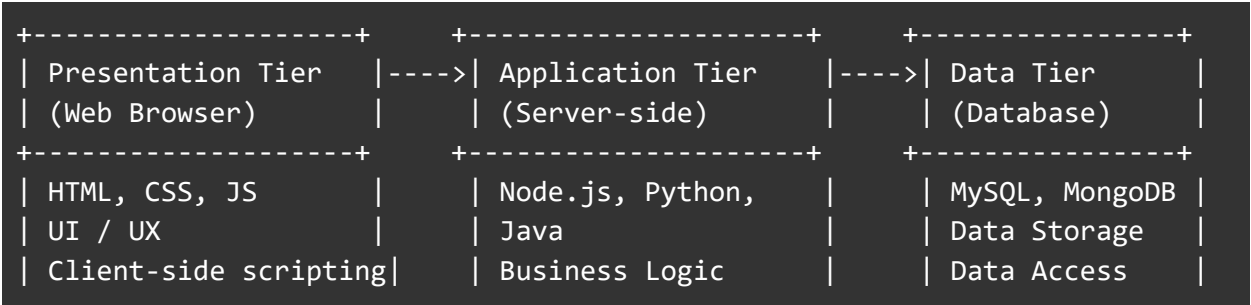
Separates an application into three layers for **scalability and maintainability**:

- **Presentation Tier (Frontend):** User interface (HTML/CSS/JS).
- **Application Tier (Backend):** Handles business logic (Node.js, Python, Java).
- **Data Tier (Database):** Stores/retrieves data (MySQL, MongoDB).

Table: Tier Overview

Tier	Purpose	Examples / Technologies	Key Responsibilities
Presentation	User interface	HTML, CSS, JS, React, Angular	UI design, UX, client-side logic
Application	Business logic	Node.js, Python, Java	APIs, data processing, security
Data	Data storage	MySQL, MongoDB, Redis	Store/retrieve data, manage database

Diagram: Three-Tier Architecture



Web Development Roles

Frontend Developer

Focus: Visual and interactive parts of websites

Responsibilities:

- Build responsive, visually appealing user interfaces.
- Convert designs from Figma/Adobe XD into code.
- Ensure cross-browser/device compatibility.
- Optimize performance for faster page loads.

Core Skills: HTML, CSS, JavaScript, React/Angular/Vue.js

Diagram: Frontend Workflow



Backend Developer

Focus: Server-side operations and databases

Responsibilities:

- Write server-side business logic.
- Handle authentication, authorization, and security.
- Connect frontend to databases and APIs.
- Optimize performance and scalability.

Core Skills: Python, Java, Node.js, SQL, REST APIs

Example: Node.js Express server fetching data from MongoDB

Full-stack Developer

Handles **both frontend and backend**

Responsibilities:

- Build end-to-end solutions.
- Coordinate database, server, and UI/UX development.

Core Skills:

- Frontend: HTML/CSS/JS/React
 - Backend: Node.js/Python/Java
-

Setting Up the Development Environment

Install a Code Editor

- **Visual Studio Code (VS Code):** Free, extensible
- Alternatives: Sublime Text, Atom, JetBrains IDEs

Install a Web Browser with Developer Tools

- **Google Chrome:** Inspect, debug, monitor network
- **Mozilla Firefox:** Alternative with robust tools

Set Up Version Control (Git)

- **Git:** Tracks code changes
- **GitHub/GitLab:** Host and collaborate on repositories

Example Commands:

```
git init                # Initialize repository
git add .               # Stage changes
git commit -m "Initial commit" # Commit changes
git push origin main    # Push to remote
```

Install Node.js and NPM

- **Node.js:** JavaScript runtime
- **NPM:** Package manager

Verify Installation:

```
node -v    # Node.js version
npm -v     # NPM version
```

Use a Local Development Server

- Option 1: VS Code Live Server extension
- Option 2: Node.js/Express server

Project Setup Example:

1. Install VS Code
2. Install Live Server extension
3. Create project folder
4. Add `index.html`
5. Right-click → Open with Live Server

Browser opens page; changes update automatically

Practical Example: Simple Web Page

HTML (`index.html`):

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Welcome to Web Development!</h1>
  <button id="clickBtn">Click Me</button>
  <script src="script.js"></script>
</body>
</html>
```

CSS (**style.css**):

```
body {  
  font-family: Arial;  
  text-align: center;  
  margin-top: 50px;  
}  
button {  
  padding: 10px 20px;  
  background-color: #007bff;  
  color: white;  
  border: none;  
  border-radius: 5px;  
}
```

JavaScript (**script.js**):

```
document.getElementById("clickBtn").addEventListener("click", function() {  
  alert("Button clicked!");  
});
```

Outcome: Clicking the button shows a **popup message**, demonstrating frontend interactivity.